

Cifrado y hashing de evidencia en la bodycam

Estado del análisis · BodyCamServer (Unisoc SC9832e, Android 9) · 20 de agosto de 2026

1 LA PREGUNTA Y LA RESPUESTA

Pregunta del cliente: ¿deben hacerse el hashing y el cifrado **en la propia bodycam**, antes de subir el vídeo al servidor?

Respuesta: sí, es viable. El coste de CPU no es un obstáculo. Medido sobre la unidad real (no estimado) con una grabación real de 20 min / 473 MB: cifrar y hashear cuesta **~5,1 s** frente a los **~3 min** que tarda la subida. El cifrado es en torno al 3 % del tiempo total del proceso.

2 TIEMPOS MEDIDOS EN LA UNIDAD

Operación	Tiempo	Velocidad	Nota
SHA-256 del fichero	1,36 s	349 MB/s	Coincide con sha256sum en PC ✓
AES-256-GCM a fichero	3,71 s	128 MB/s	Incluye escritura en disco
AES-256-GCM, techo de CPU	—	170 MB/s	Sin I/O
Total hash + cifrado	~5,1 s	—	Vídeo de 20 min (473 MB)
Subida a 20 Mbps (referencia)	~3 min	2,5 MB/s	~35× más lenta que el cifrado

Sobrecoste de tamaño del cifrado: +0,0015 % (nonce y tag por bloque). Irrelevante.

3 RESTRICCIONES TÉCNICAS QUE CONDICIONAN LA IMPLEMENTACIÓN

- **AES/GCM no admite streaming en Android.** La pila del sistema acumula todo el fichero en memoria y solo cifra al final: con 473 MB se queda sin memoria. Solución adoptada: trocear en **bloques de 1 MB independientes**, cada uno con su nonce y su tag.
- **El cifrado se dispara al cerrar la grabación**, nunca en paralelo: la unidad ya se calienta grabando.
- **El script aes-256Sha256.py que envió el cliente no sirve como medida de referencia:** usa un único nonce y un único tag para todo el fichero (patrón inviable en la unidad), corre sobre Python + OpenSSL — otra pila distinta a la del dispositivo — y la unidad no tiene Python instalado.

Hueco abierto: hoy **no existe ruta de descifrado en ninguna parte** — ni en la app ni en el servidor. Lo medido y construido solo cifra. Sin definir quién descifra, dónde y con qué clave, la evidencia cifrada no es recuperable.

4 SUBIDA DE LA EVIDENCIA (CADENA DE CUSTODIA)

Situación actual: el servicio de subida envía el fichero entero de una vez, en una sola petición. Sin reanudación: un corte a los 2 GB de 2,2 GB obliga a repetir la subida completa.

Propuesta evaluada: subida por trozos reanudable con el protocolo **tus 1.0.0**. Investigación cerrada, **sin implementar**.

- **OJO tus da reanudación, no integridad.** La librería no implementa la verificación de checksum, aunque su documentación lo sugiera. El hash hay que declararlo aparte y **verificarlo en el servidor** al ensamblar los trozos.
- **Recomendación:** no implementar el protocolo dentro de Nexus, sino poner **tusd** (el servidor de referencia, ya hecho) delante, con un hook que llame a la ingestión existente al terminar la subida.

5 LO QUE NECESITAMOS SABER DEL BACKEND

Las decisiones que quedan **no son técnicas de la app**: dependen del servidor y del cliente, y bloquean el desarrollo.

1. ¿Qué debe poder hacer el servidor sin tener la clave?

Si solo almacena → ciframos el fichero entero, simple. Si debe listar, previsualizar o verificar el vídeo por su cuenta → hace falta cifrado selectivo de las muestras de vídeo (estándar ISO cenc / cbcs) dejando el contenedor en claro; es un desarrollo bastante mayor. **Esta respuesta define toda la arquitectura.**

2. ¿Contra quién protege el cifrado?

Proteger frente a quien extrae la tarjeta de memoria no es lo mismo que proteger frente a quien opera el servidor. Determina dónde vive la clave: en el almacén seguro del propio dispositivo, o una clave por incidente envuelta con la clave pública del servidor.

3. ¿Quién descifra la evidencia, dónde y con qué proceso?

Hay que construirlo; hoy no existe. ¿Descifra el backend al ingerir, o se conserva cifrada en reposo y descifra el visor?

4. ¿El servidor verificará el SHA-256 al recibir, y sobre qué: el fichero en claro o el cifrado?

Es lo que sostiene la cadena de custodia. Hay que acordar cómo se declara el hash al subir y qué hace el servidor si no cuadra.

5. ¿Qué tecnología es `nexus.aeriaone.com` y tenemos acceso a su repositorio?

Decide si vamos con tusd delante o con la librería nativa del framework. Necesitamos además la URL contra la que probar la subida por trozos.

Los números proceden de medición directa sobre la unidad física (endpoint de benchmark ejecutado en el propio dispositivo), no de estimaciones. Existe un informe técnico completo con el detalle de las mediciones, el análisis del script del cliente y el plan de implementación en 6 pasos.